



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE CIÊNCIAS EXATAS E DA TERRA
DEPARTAMENTO DE INFORMÁTICA E MATEMÁTICA APLICADA
PROGRAMA DE PÓS-GRADUAÇÃO EM SISTEMAS E COMPUTAÇÃO



**REPRODUÇÃO DE EXPERIMENTOS DE MEDIÇÃO DE ENERGIA EM
LINGUAGENS DE PROGRAMAÇÃO: C++ E PYTHON**

Maycon Nazario de Freitas

Prof. Dr. Sergio Medeiros
Natal – RN
2026

1 Introdução

A relação entre a linguagem de programação e o consumo de energia do software é objeto de uma linha de trabalhos que culmina no estudo de Pereira et al. (2021), que medem tempo, energia e memória de programas do Computer Language Benchmarks Game (Gouy, 2026) em 27 linguagens e as ordenam por eficiência energética. O ranking resultante, em que, por exemplo, Python consome 75,88 vezes a energia de C, foi citado em palestras da indústria como argumento para a escolha de linguagem. Van Kempen et al. (2024) respondem a esse trabalho com um modelo causal: distinguem a linguagem de sua implementação, identificam o número de núcleos ativos e a atividade de memória como os fatores que governam a potência, corrigem erros do medidor original e concluem que, controlados esses fatores, a escolha da linguagem não tem efeito sobre a energia além do que tem sobre o tempo de execução.

Este relatório reproduz os experimentos dos dois artigos, restritos a C++ e Python, na mesma máquina, e compara os resultados obtidos com os publicados. A restrição a duas linguagens deixa fora as anomalias entre linguagens que os artigos discutem (C e C++, JavaScript e TypeScript, Java, Go) e mantém o par que separa a maior distância do ranking: uma linguagem compilada e uma interpretada.

1.1 Pereira et al. (2021): protocolo e adaptações

O estudo de Pereira et al. (2021) mede, para dez problemas do Benchmarks Game, a versão mais rápida disponível em cada linguagem. Cada programa é executado dez vezes por um medidor em C que lê os contadores RAPL antes e depois de cada execução. A energia reportada é a soma dos domínios CPU e DRAM, e o tempo é o da própria chamada. As medições foram feitas em um Intel Core i5-4460 (Haswell, quatro núcleos, 3,2 GHz) com Ubuntu Server 16.10 e núcleo Linux 4.8. O artigo menciona um intervalo de dois minutos entre execuções e o descarte dos 20% de valores mais baixos e mais altos antes de calcular a média.

A reprodução usou o repositório publicado pelos autores, restrito às linguagens C++ e Python. Os programas, o `compile_all.py`, o `main.c` e a lógica de medição são os originais; a campanha foi lançada pelo `compile_all.py` dos autores, que compila e mede cada benchmark dez vezes. Quatro adaptações foram necessárias para o ambiente atual. Os 20 Makefiles trocam `/usr/bin/g++` por `clang++-19` e o caminho de `python3.6` por `python3`; o `gen-input.sh` troca `python` por `python3`; o `fasta.gpp-5.c++` recebeu um `#include <functional>` para compilar com Clang 19; e o `rapl.c` teve quatro linhas alteradas para incluir o modelo 158 do processador (Coffee Lake, identificado no código como Kaby Lake) nas condições que leem os domínios GPU e DRAM, sem o que o medidor gravaria só PACKAGE e CORE. O intervalo de dois minutos entre execuções descrito no artigo não está implementado no `compile_all.py` e não foi acrescentado. O medidor grava, por execução, os domínios PACKAGE, CORE, GPU e DRAM e o tempo em milissegundos. A campanha rodou em um contêiner Docker construído a partir do Dockerfile do repositório desta atividade, com Clang

19 e CPython 3.12.6, as versões fixadas por van Kempen et al. (2024) e adotadas nas duas reproduções para permitir a comparação da Seção 4.

1.2 van Kempen et al. (2024): protocolo e adaptações

O estudo de van Kempen et al. (2024) parte das mesmas implementações usadas por Pereira et al. (2021). O artigo mede 13 implementações de 11 linguagens em dois processadores Intel Xeon Gold 6430, com 128 núcleos lógicos e 128 GB de memória, Linux 6.8.0-45, dentro de contêineres Docker, e estende a avaliação a duas suítes adicionais, SPEC CPU 2017 e DaCapo.

O artigo usa duas configurações. Na configuração padrão, o contêiner tem acesso a todos os núcleos e o processador opera com seu escalonamento de frequência normal; é a configuração das Figuras 8a e 8b do artigo, que relacionam a potência do pacote ao número médio de núcleos ativos e a potência de DRAM à taxa de faltas de LLC. Na segunda, o processador é fixado na frequência mínima com o turbo desligado e o contêiner é restrito a um único núcleo por `--cpuset-cpus`; é a configuração do resultado central de RQ1, em que todos os pares apresentam a mesma potência, de $189,8 \pm 0,5$ W, e a energia se torna proporcional ao tempo. Nessa configuração os dados publicados têm três execuções por par.

A reprodução usou o repositório publicado pelos autores, com uma única alteração nos arquivos: o `docker/main.Dockerfile` foi cortado para instalar apenas as cadeias de C++ e Python. Três desvios de procedimento foram necessários. O `main.Dockerfile` parte de `ubuntu:latest`, que hoje resolve para a versão 26.04, sem pacotes do LLVM 19, e a imagem foi construída com `--build-context` fixando `ubuntu:24.04`; a ferramenta RAPL dos autores foi compilada dentro da imagem, porque o host não tem `cmake`; e as entradas de fasta foram geradas em um contêiner `node:20-slim` descartável. A imagem resultante tem Clang 19.1.7 e CPython 3.12.6, que correspondem às versões da Tabela V do artigo, e o pacote `gmpy2` na versão 2.2.1, fixada no `requirements.txt` dos autores. Medidor, Makefiles, entradas e o script de campanha `measure.py` são os originais. A campanha de frequência mínima seguiu o README dos autores: turbo desligado, governador `powersave`, contêiner restrito ao núcleo 7, uma execução de aquecimento e três medidas.

1.3 Ambiente, dados e convenções de análise

As duas reproduções rodaram na mesma máquina, um Intel Core i7-9700 (Coffee Lake, oito núcleos, sem SMT, limites de frequência de 800 MHz e 4,7 GHz) com Ubuntu 24.04.4 LTS e núcleo Linux 6.8.0-136-generic, em contêineres Docker com o mesmo compilador e o mesmo interpretador, em três estados do processador: 3,0 GHz sem turbo com todos os núcleos, na campanha de Pereira et al. (2021); 4,7 GHz com turbo e todos os núcleos, na campanha padrão de van Kempen et al. (2024); e 800 MHz em um único núcleo, na campanha de frequência mínima. O código das duas campanhas, os dados brutos medidos e os

scripts que geram todas as tabelas e figuras deste relatório estão no repositório do GitHub (<https://github.com/maycon-freitas/reproduction-energy-languages>).

Em todo o relatório, “restrito” designa os dados brutos publicados pelos autores de cada artigo, medidos na máquina deles e restritos a C++ e Python, “reprodução” os dados medidos nesta atividade e “artigo” os valores impressos nos artigos. Cada reprodução usa o domínio RAPL e o método de agregação do artigo correspondente: CORE mais DRAM, o domínio que a Seção 2.1 identifica, e média aparada a 20%, o método declarado no artigo, para Pereira et al. (2021); PKG mais DRAM ou PKG e mediana, os dos scripts de análise dos autores, para van Kempen et al. (2024). No i7-9700, CORE corresponde a 95% a 99% de PKG na campanha de Pereira et al. (2021) e a 98% a 100% na configuração padrão de van Kempen et al. (2024), de modo que a diferença de domínio entre as duas seções é menor que qualquer das diferenças discutidas.

Além das medidas que os artigos usam, a análise emprega o coeficiente de variação entre execuções, como medida de estabilidade.

1.4 Benchmarks e paralelismo das implementações

Os dois artigos medem as mesmas implementações de dez problemas do Benchmarks Game, a versão mais rápida disponível em cada linguagem, e nenhum dos dois publica quantos núcleos cada implementação ocupa. Essa informação decide boa parte da análise, porque a potência do pacote depende do número de núcleos ativos, e van Kempen et al. (2024) apontam sua ausência como uma das causas das anomalias nos resultados de Pereira et al. (2021). A Tabela 1 registra, para cada benchmark e linguagem, o mecanismo de paralelismo encontrado no código e o número médio de núcleos ativos medido pelo `task-clock` na campanha padrão de van Kempen et al. (2024), na máquina de reprodução, com todos os oito núcleos disponíveis.

Tabela 1: Paralelismo das implementações: mecanismo encontrado no código-fonte e número médio de núcleos ativos medido pelo `task-clock`, mediana de 21 execuções na configuração padrão de van Kempen et al. (2024). `fasta` não tem versão C++ no repositório de van Kempen et al. (2024), e `pidigits` fica fora das duas reproduções (Seções 2.1 e 3.1).

Benchmark	C++		Python	
	mecanismo	núcleos	mecanismo	núcleos
binary-trees	OpenMP	5,94	multiprocessing	6,52
fannkuch-redux	OpenMP	7,96	multiprocessing	7,88
fasta	threads	–	nenhum	1,00
k-nucleotide	threads	5,43	multiprocessing	7,65
mandelbrot	OpenMP	7,94	multiprocessing	7,98
n-body	só flag	1,00	nenhum	1,00
regex-redux	OpenMP	1,69	multiprocessing	2,26
reverse-complement	threads	1,67	multiprocessing	1,49
spectral-norm	OpenMP	7,89	multiprocessing	3,99

O mecanismo vem da inspeção do código: em C++, a presença de diretivas `#pragma omp` ou de criação de threads no fonte, ou, na ausência delas, das flags `-fopenmp` e `-pthread` no Makefile (caso registrado como “só flag”); em Python, a importação do módulo `multiprocessing`. O código indica a intenção; o contador mede o resultado, e os dois discordam em três casos. `regex-redux` em C++ tem diretivas OpenMP no fonte, mas ocupa 1,7 núcleos, porque a execução é dominada pela biblioteca de expressões regulares; `reverse-complement` em Python importa `multiprocessing`, mas ocupa 1,5; `n-body` em C++ compila com `-fopenmp` sem diretiva no código e ocupa 1,0. Nos demais casos código e medida concordam.

A divisão entre implementações paralelas e sequenciais se dá por benchmark e é a mesma nas duas linguagens, salvo `spectral-norm`, que ocupa 7,9 núcleos em C++ e 4,0 em Python. `fasta` em Python é sequencial; a versão C++ não foi medida nesta campanha.

2 Reprodução de Pereira et al. (2021)

2.1 Validação do método com os dados restritos

Antes de comparar as medições, o pipeline de análise foi aplicado aos arquivos CSV publicados pelos autores, para reproduzir as tabelas do artigo a partir dos dados brutos. Esse passo revelou duas particularidades do artefato e dois problemas nos dados.

A primeira é a composição da energia. O artigo descreve a coluna *Energy* como a soma de CPU e DRAM, sem indicar o domínio RAPL correspondente. O medidor grava PACKAGE e CORE como colunas separadas, e a soma que reproduz os valores da Tabela 3 do artigo é CORE mais DRAM: ela devolve 41,25 J para `binary-trees` em C++ (o artigo reporta 41,23 J), 219,91 J para `fannkuch-redux` (219,89 J) e 34,94 J para `fasta` (34,88 J), enquanto PACKAGE mais DRAM daria valores de 15% a 25% maiores. A distinção importa porque a relação entre CORE e PACKAGE depende do processador: no i5-4460 o CORE corresponde a 55% a 89% do PACKAGE, conforme o benchmark; no i7-9700, a 95% a 99%. Todas as comparações desta seção usam CORE mais DRAM. Van Kempen et al. (2024) medem o domínio PKG, diferença que a Seção 4 leva em conta.

Dois problemas dos artefatos afetam os dados e precisam ser tratados antes das tabelas. O primeiro é consequência da ausência de verificação da saída, mantida na reprodução como no medidor original. A versão de `pidigits` em Python importa `div` do pacote `gmpy2` e depende do comportamento da versão 2.0 dessa biblioteca, em que `div` é divisão inteira. A partir do `gmpy2` 2.1, `div` passou a ser divisão real e devolve um `mpfr`; a imagem da reprodução instalou a versão 2.3.1, sem versão fixada no `Dockerfile`, e o programa executa sem erro, mas produz dígitos incorretos e leva cerca de 360 s em vez de pouco mais de 1 s. A troca de `div` por `f_div` na linha de importação restaura a saída correta e o tempo esperado. Como a correção não foi aplicada antes da campanha, a medição de `pidigits` em Python é inválida, e o benchmark foi

retirado de todas as tabelas e figuras desta seção, nas duas linguagens e nos dois conjuntos de dados; a Seção 3.1 mostra que o mesmo defeito está nos dados publicados por van Kempen et al. (2024). Os resultados a seguir cobrem os nove benchmarks restantes, que coincidem com os nove que o próprio artigo usa em sua Tabela 4.

O segundo é um defeito do medidor, e afeta os dois conjuntos. Os contadores RAPL têm 32 bits e a ferramenta dos autores lê apenas o valor inicial e o final, convertidos para joules antes da subtração; quando o contador dá a volta durante uma execução, o resultado é negativo. Van Kempen et al. (2024) apontam esse defeito, junto com a inclusão dos bits reservados do registrador, como erros do medidor, e observam leituras negativas nos arquivos publicados. A reprodução confirma o diagnóstico: os dados restritos contêm três leituras negativas e a reprodução, uma. A análise recupera o valor somando a capacidade do contador, 262 144 J, em vez de descartar a execução. O estouro não depende da duração da execução: ocorre a cada 262 144 J acumulados no contador e atinge a execução em curso naquele momento, com probabilidade proporcional à energia dela. As leituras negativas estão em *n-body*, *mandelbrot* e *fasta* em Python nos dados restritos, e em *spectral-norm* em Python na reprodução.

2.2 Energia e tempo por benchmark

As Tabelas 2 e 3 apresentam, no formato da Tabela 3 do artigo, a energia, o tempo e a razão entre os dois para cada benchmark, nos dados restritos e na reprodução. A razão corresponde à potência média, em kW. As Figuras 1 e 2 mostram os mesmos dados no formato das Figuras 7 a 9 do artigo, com a energia de CPU e de DRAM empilhadas, o tempo em linha e a razão anotada, uma figura por linguagem; em cada uma, o painel superior traz os dados restritos e o inferior a reprodução, com os mesmos limites nos eixos de energia e de tempo.

Tabela 2: Energia, tempo e razão energia/tempo por benchmark, para C++ e Python, nos dados restritos de Pereira et al. (2021).

Benchmark	C++			Python		
	Energy (J)	Time (ms)	Ratio (J/ms)	Energy (J)	Time (ms)	Ratio (J/ms)
binary-trees	41,25	1107,69	0,037	1794,43	44837,17	0,040
fannkuch-redux	219,91	6116,33	0,036	12788,56	279542,17	0,046
fasta	34,94	1152,86	0,030	1059,28	74059,35	0,014
k-nucleotide	111,10	3813,73	0,029	1600,77	38924,98	0,041
mandelbrot	32,44	857,45	0,038	7785,70	162831,00	0,048
n-body	57,29	3770,26	0,015	7873,79	558883,33	0,014
regex-redux	175,42	10610,18	0,017	161,63	7126,69	0,023
reverse-complement	6,16	222,54	0,028	32,26	1492,69	0,022
spectral-norm	17,23	665,46	0,026	6422,20	137717,17	0,047

Antes de interpretar as diferenças, é preciso verificar que elas não são ruído. O coeficiente de variação das dez execuções, medida que o artigo não publica, fica na reprodução abaixo de 2% em 15 das 18 combinações de benchmark e linguagem para a energia, com máximo de

Tabela 3: Energia, tempo e razão energia/tempo por benchmark, para C++ e Python, na reprodução.

Benchmark	C++			Python		
	Energy (J)	Time (ms)	Ratio (J/ms)	Energy (J)	Time (ms)	Ratio (J/ms)
binary-trees	28,42	1150,01	0,025	590,90	18447,93	0,032
fannkuch-redux	167,28	4883,81	0,034	4683,07	121483,67	0,039
fasta	19,97	1051,38	0,019	316,31	37732,33	0,008
k-nucleotide	47,62	1927,18	0,025	789,24	21337,23	0,037
mandelbrot	16,51	474,80	0,035	2972,77	71633,87	0,041
n-body	19,69	2752,86	0,007	2237,42	282761,50	0,008
regex-redux	103,78	10617,23	0,010	88,58	7254,04	0,012
reverse-complement	3,93	315,84	0,012	19,96	1779,23	0,011
spectral-norm	5,75	218,01	0,026	1855,25	80125,95	0,023

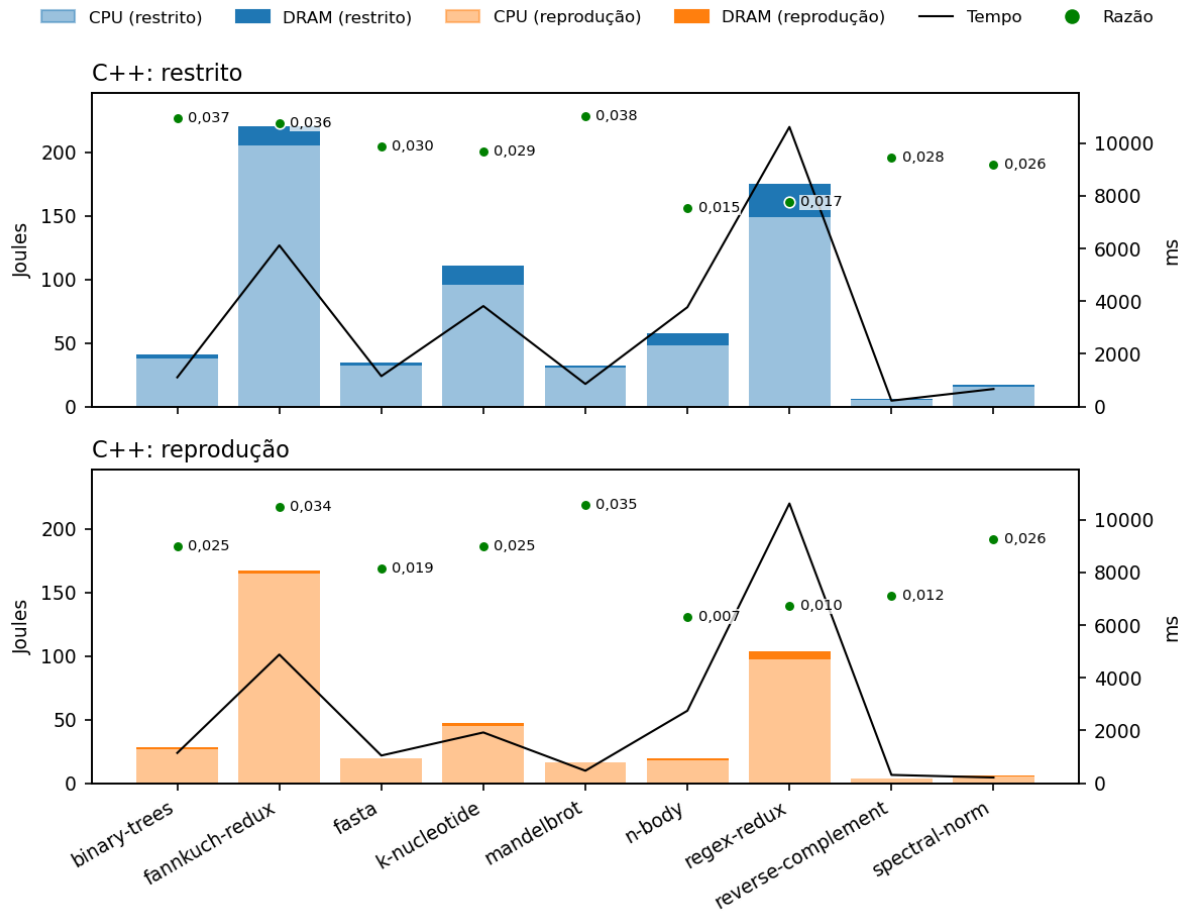


Figura 1: Energia de CPU e DRAM, tempo e razão por benchmark em C++, dados restritos (acima) e reprodução (abaixo), na mesma escala.

4,6% (reverse-complement em C++), e abaixo de 2% em 16 delas para o tempo, com máximo de 4,2%. Nos dados restritos a dispersão é semelhante na maioria dos casos, com os pontos fora da faixa concentrados em reverse-complement (16,7% na energia e 117% no tempo em C++; 25,6% no tempo em Python) e em k-nucleotide em C++ (5,8% e 11,8%). A reprodução é, portanto, ao menos tão estável quanto o conjunto restrito, e as diferenças discutidas a seguir são

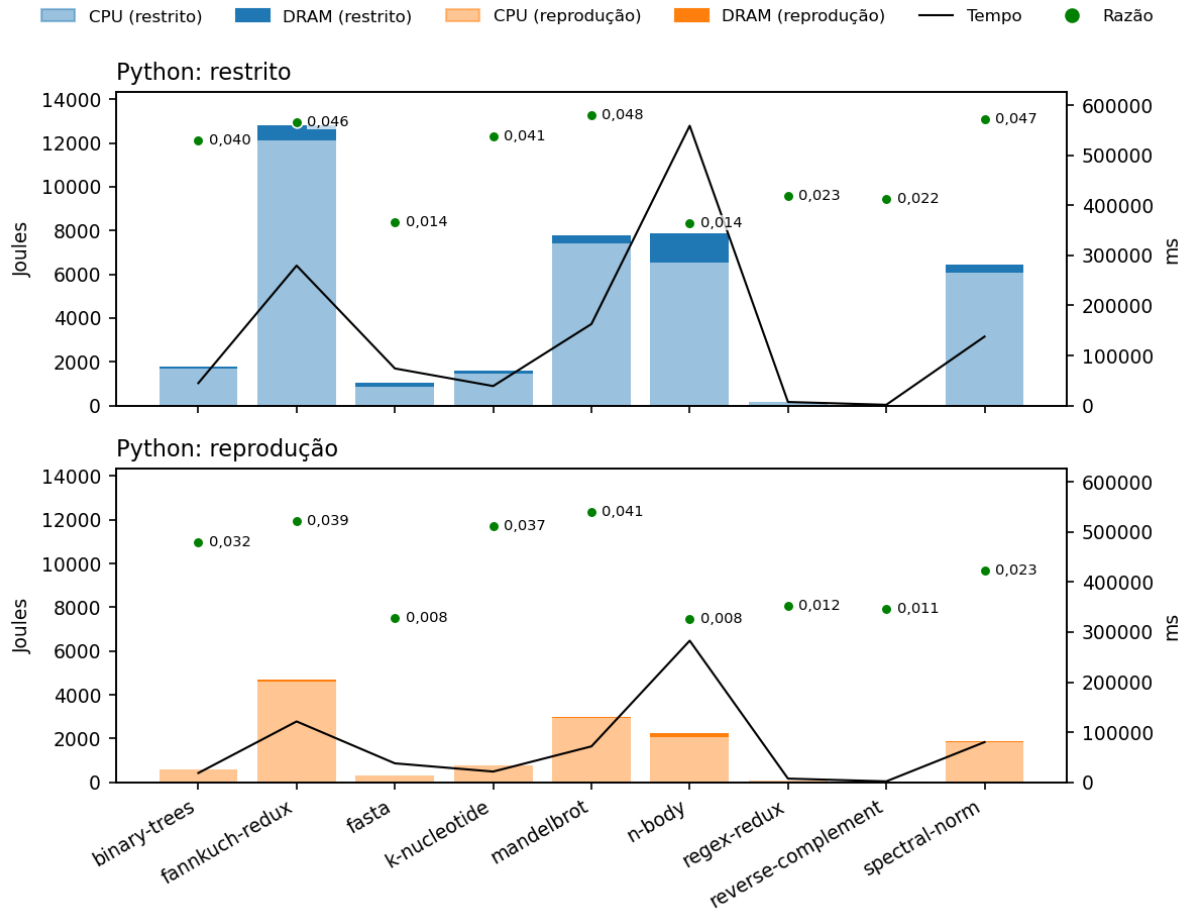


Figura 2: Energia de CPU e DRAM, tempo e razão por benchmark em Python, dados restritos (acima) e reprodução (abaixo), na mesma escala.

maiores que a variação entre execuções por uma ordem de grandeza ou mais.

Em valores absolutos, a reprodução consumiu menos energia que o conjunto restrito em todos os benchmarks e nas duas linguagens, como se espera de um processador cinco anos mais novo. O tempo, porém, não caiu na mesma proporção, e a forma como energia e tempo se distribuem entre linguagens e benchmarks é o que interessa. O tempo de Python caiu para 41% a 58% do conjunto restrito nos benchmarks de cálculo (binary-trees, fannkuch-redux, fasta, k-nucleotide, mandelbrot, n-body, spectral-norm) e ficou igual ou maior em regex-redux e reverse-complement, dominados por entrada e saída e por bibliotecas em C. O tempo de C++ caiu para um terço em spectral-norm e para metade em mandelbrot e k-nucleotide, três dos benchmarks paralelos da Tabela 1, e ficou igual ou maior em binary-trees, regex-redux e reverse-complement. Nesses três, a Figura 1 e as Tabelas 2 e 3 mostram a energia da reprodução 30% a 40% mais baixa com tempo igual ou maior: o programa levou o mesmo tempo consumindo menos potência.

Quatro fatores explicam esse padrão, e nenhum deles é a linguagem. O primeiro é a versão do interpretador: o artigo usou CPython 3.6.1 e a reprodução usou CPython 3.12.6, que incorpora as otimizações introduzidas a partir da versão 3.11, citadas por van Kempen

et al. (2024) como fonte de ganho relevante. Isso explica por que Python ganhou cerca de duas vezes nos benchmarks em que o tempo é gasto no interpretador e nada nos benchmarks em que o trabalho é feito por código em C dentro do runtime. O segundo é o paralelismo das implementações: os maiores ganhos de C++, de um terço a metade do tempo, estão em `spectral-norm`, `mandelbrot` e `k-nucleotide`, que ocupam de 5,4 a 7,9 núcleos (Tabela 1) e aproveitaram o dobro de núcleos disponíveis. O terceiro é o compilador de C++, do g++ usado pelo artigo para o Clang 19. O quarto é a frequência: a máquina de reprodução operou a 3,0 GHz sem turbo, contra os 3,2 GHz nominais do i5-4460, cujo estado de turbo o artigo não relata.

2.3 Energia de DRAM

A Tabela 4 apresenta a energia de DRAM de cada benchmark, nos dados restritos e na reprodução, com a média na última linha, e a Tabela 5 apresenta essa média junto com a fração da energia total atribuída à CPU, nos moldes da Tabela 6 do artigo. A fração é a energia de CPU dividida pela soma de CPU e DRAM, calculada por benchmark e depois tirada a média das nove. A energia de DRAM caiu mais que a de CPU: a média por benchmark foi de 8,54 J no artigo para 1,79 J na reprodução em C++, e de 358,75 J para 44,8 J em Python, reduções de 4,8 e 8,0 vezes, enquanto a energia de CPU caiu 1,6 e 2,8 vezes. Como a DRAM encolheu mais que a CPU, a fração da energia total atribuída à CPU subiu de 89% para 95% nas duas linguagens, embora a CPU também tenha consumido menos em valor absoluto.

Tabela 4: Energia de DRAM por benchmark (J), para C++ e Python, nos dados restritos e na reprodução, com a média dos nove benchmarks.

Benchmark	restrito		reprodução	
	C++	Python	C++	Python
binary-trees	3,7	127	1,3	25,8
fannkuch-redux	14,8	675	2,8	69,0
fasta	2,9	179	0,7	21,5
k-nucleotide	15,2	121	2,4	33,4
mandelbrot	2,1	393	0,3	40,8
n-body	9,1	1350	1,6	161
regex-redux	26,1	18,0	6,5	5,0
reverse-complement	0,9	4,3	0,4	1,6
spectral-norm	1,6	333	0,1	45,7
média	8,5	356	1,8	44,8

O que se reproduz é a ordem e o padrão: Python consome mais DRAM que C++ em todos os benchmarks de cálculo, e os benchmarks que mais gastam DRAM são os mesmos nos dois conjuntos (`n-body`, `fannkuch-redux` e `spectral-norm` em Python; `regex-redux`, `k-nucleotide` e `fannkuch-redux` em C++). O que não se reproduz é a proporção. A afirmação do artigo de que a CPU responde por cerca de 89% da energia, com o restante em DRAM, e

Tabela 5: Energia média de DRAM e fração da energia consumida pela CPU, na reprodução e no artigo.

Origem	Linguagem	DRAM média (J)	CPU (%)
reprodução	C++	1,79	95,13
	Python	44,82	95,38
artigo	C++	8,54	88,94
	Python	358,75	87,98

de que essa proporção se mantém entre tipos de linguagem, é específica da plataforma original: a fração depende da geração do processador e da memória e do domínio RAPL escolhido para representar a CPU, e no i7-9700 fica em 95%.

2.4 Resultados agregados e correlação entre energia e tempo

A Tabela 6 reproduz o critério da Tabela 4 do artigo: média dos nove benchmarks, normalizada pela linguagem mais eficiente. Como o artigo normaliza por C, que está fora do escopo, os valores publicados foram convertidos para a base C++.

Tabela 6: Médias de energia e tempo, absolutas e normalizadas por C++, na reprodução e no artigo; os valores normalizados do artigo foram convertidos da base C para a base C++.

Origem	Linguagem	Energy média (J)	Time médio (ms)	Energy normalizada	Time normalizado
reprodução	C++	45,88	2599,01	1,00	1,00
	Python	1505,95	71395,08	32,82	27,47
artigo	C++	77,50	3155,00	1,00	1,00
	Python	4390,00	145178,00	56,63	46,09

A posição relativa das duas linguagens se mantém e a magnitude não. No artigo, Python consome 56,6 vezes a energia de C++ e leva 46,1 vezes o tempo; na reprodução, 32,8 e 27,5. O fator de 75,88 em energia para Python, que van Kempen et al. (2024) registram ter sido citado em palestras da indústria como argumento de escolha de linguagem, é um número de protocolo: com as mesmas implementações, em hardware mais novo e com o interpretador atual, a distância entre as duas linguagens cai para pouco mais da metade. O que caiu não foi a distância entre duas linguagens, mas a distância entre duas versões de runtime medidas em duas máquinas, como a Seção 2.2 mostrou.

A Tabela 7 reproduz a Tabela 5 do artigo para as duas linguagens: correlação de Spearman entre energia e tempo ao longo dos benchmarks, com a classificação de Rea e Parker adotada pelo artigo. O artigo calcula sobre dez benchmarks; a reprodução, sobre nove, e a tabela traz uma terceira linha com os dados restritos aos mesmos nove, para separar o efeito do conjunto do efeito da medição. Os dados restritos com nove benchmarks dão 0,97 e 0,93, os mesmos valores do artigo; a reprodução dá 0,87 nas duas linguagens. A correlação permanece

na classe “muito forte”, e a queda é da medição. Ela é coerente com a maior dispersão de potência na reprodução: a razão energia/tempo varia por um fator de 3,4 entre benchmarks no conjunto restrito (0,014 a 0,048 J/ms) e de 5,9 na reprodução (0,007 a 0,041 J/ms), porque na máquina de reprodução a diferença de potência entre um núcleo e oito é maior do que a que os dados restritos mostram. Quanto maior a variação de potência entre benchmarks, menos a energia acompanha o tempo.

Tabela 7: Correlação de Spearman entre energia e tempo por linguagem, com a classificação de Rea e Parker adotada pelo artigo: reprodução e dados restritos sobre nove benchmarks, artigo sobre dez.

Origem	Linguagem	Spearman E x T	Classificação
reprodução	C++	0,87	muito forte
	Python	0,87	muito forte
restrito	C++	0,97	muito forte
	Python	0,93	muito forte
artigo	C++	0,97	muito forte
	Python	0,93	muito forte

Esse resultado diz respeito à RQ2 do artigo, se a linguagem mais rápida é sempre a mais econômica. O artigo responde que não, com dois argumentos: a variação da razão energia/tempo entre linguagens e os casos em que a ordem por energia difere da ordem por tempo. No par C++ e Python, o segundo argumento não encontra exemplo: nos nove benchmarks, nos dois conjuntos, a linguagem mais rápida é também a que consome menos energia, incluindo *regex-redux*, onde Python vence nas duas métricas. O primeiro argumento se reproduz, mas a variação acompanha o número de núcleos que cada implementação usa (Tabela 1), não a linguagem. A resposta negativa do artigo a RQ2 está correta na forma e mal atribuída na causa: a potência não é constante, mas o que a faz variar é o paralelismo da implementação.

3 Reprodução de van Kempen et al. (2024)

3.1 Validação do método com os dados restritos

Como na Seção 2.1, o pipeline de análise foi aplicado primeiro aos arquivos JSON publicados pelos autores, restritos a C++ e Python, para verificar se reproduz os valores do artigo. Os dados exigiram três ajustes de escopo, e a comparação com o artigo exige uma decisão sobre os valores de referência.

O primeiro ajuste é o conjunto de pares. O repositório dos autores não contém a implementação de *fasta* em C++; é um dos 12 pares que o artigo declara ter omitido por falta de código-fonte ou por falha de compilação ou execução. A configuração padrão dos dados publicados inclui ainda quatro programas da suíte SPEC em C++, que ficam fora desta análise, restrita ao Benchmarks Game.

O segundo é o formato dos dados. A máquina dos autores tem dois soquetes, e cada amostra de energia traz uma leitura de PKG e uma de DRAM por soquete; a análise soma as duas, como fazem os scripts dos autores. O Xeon não expõe o domínio PP0, que é o domínio dos núcleos, o CORE da Seção 2, gravado como zero nos dados publicados; o i7-9700 o expõe, e na reprodução PP0 corresponde a 98% a 100% de PKG na configuração padrão e a 78% a 84% na frequência mínima. Essa relação é a que permite cruzar as medições desta seção, feitas em PKG, com as da Seção 2, feitas em CORE.

O terceiro é `pidigits`. O problema descrito na Seção 2.1 se repete aqui, e desta vez nos dados publicados. O fonte é o mesmo de Pereira et al. (2021), e na versão fixada do `gmpy2`, 2.2.1, `div` devolve um `mpfr`: na imagem da reprodução, `div(7, 2)` resulta em 3,5. Nos dados publicados, `pidigits` em Python leva 322 s contra 0,72 s em C++, razão de 445, contra 2,3 nos dados de Pereira et al. (2021); na reprodução, 232 s contra 0,55 s. O artigo não identifica o erro: sua única observação sobre o benchmark é que a versão para PyPy não compila a biblioteca, e o programa em CPython executa sem erro. A inclusão não afeta os resultados do artigo, porque a potência de um programa não depende de sua saída estar correta, e o artigo não publica tempo nem energia por benchmark. Esta reprodução retira o benchmark de todas as tabelas e figuras, pela mesma razão da Seção 2. Os resultados a seguir cobrem 17 pares, oito em C++ e nove em Python, com 357 execuções por conjunto na configuração padrão e 51 na frequência mínima.

A decisão é a fonte dos valores de referência. Os três resultados numéricos do artigo que incluem C++ e Python, a potência a frequência mínima e os coeficientes das Figuras 8a e 8b, são calculados sobre as 13 implementações e as três suítes, não sobre as duas linguagens desta reprodução. Compará-los diretamente com a reprodução misturaria duas diferenças, a de escopo e a de medição. As tabelas desta seção trazem, por isso, três linhas: o valor impresso no artigo; o mesmo cálculo, com os scripts dos autores, sobre os dados publicados restritos a C++ e Python; e a reprodução. A diferença entre as duas primeiras linhas é o efeito do recorte a duas linguagens, e a diferença entre a segunda e a terceira é o efeito da máquina e da medição. Nas figuras, o painel dos dados restritos traz as duas curvas, a publicada no artigo e a ajustada ao recorte.

3.2 Potência a frequência mínima em um núcleo

O resultado central do artigo, em RQ1, é que, fixadas a frequência e o número de núcleos, todos os pares apresentam a mesma potência, de modo que a energia é proporcional ao tempo e a implementação da linguagem não tem efeito sobre a energia além do que tem sobre o tempo. A Tabela 8 resume a reprodução ao lado do valor do artigo e do mesmo cálculo sobre os dados publicados restritos a C++ e Python.

O recorte a duas linguagens não altera o resultado dos autores: $189,69 \pm 0,38$ W contra $189,8 \pm 0,5$ W. Na reprodução, as 51 execuções ficam em $2,23 \pm 0,11$ W, com dispersão de 5%, contra 0,26% no artigo. A dispersão maior é esperada: o servidor tem 128 núcleos, e o RAPL

Tabela 8: Potência (PKG mais DRAM) na frequência mínima em um núcleo, sobre todas as execuções: valor do artigo, calculado sobre 13 implementações e três suítes; dados restritos a C++ e Python; e reprodução. O artigo não publica os extremos.

Origem	Média (W)	Desvio (W)	CV (%)	Mínimo (W)	Máximo (W)	Execuções
artigo	189,80	0,50	0,26	–	–	–
restrito	189,69	0,38	0,20	189,12	191,00	51
reprodução	2,23	0,11	5,00	1,95	2,42	51

mede o pacote inteiro, incluindo os parados, de modo que a variação entre programas se dilui em uma potência de fundo maior que a do desktop de oito núcleos. Que a reprodução resulte em 5% sustenta a afirmação do artigo com uma reserva: a potência é a mesma para todos os pares a menos de 5%.

3.3 Potência em função do número de núcleos ativos

RQ2 do artigo identifica o número de núcleos ativos como o principal fator da potência e o quantifica por um ajuste logarítmico: $31 \cdot \log_2(x) + 246$ W, R^2 de 0,72. A leitura é que dobrar os núcleos acrescenta 31 W e que paralelizar compensa em energia sempre que o ganho de vazão supere 13%. A Figura 3 mostra as execuções no mesmo formato, com a curva do artigo e o ajuste aos dados restritos no painel superior e o ajuste da reprodução no inferior; a Tabela 9 compara os coeficientes.

O ajuste da reprodução dá $13,4 \cdot \log_2(x) + 30,4$ W, com R^2 de 0,85. A base (30,4 W) é a potência com um núcleo; o acréscimo por dobrar os núcleos (13,4 W) é menos da metade do artigo. A consequência é que paralelizar compensa a partir de 44% de ganho de vazão no desktop, não de 13%. Nos dados restritos o ajuste dá $37,5 \cdot \log_2(x) + 212,9$ W, com limiar de 18%: parte da distância entre 13% e 44% vem do recorte a duas linguagens, e o restante da máquina. O artigo ressalva que o resultado pode variar entre plataformas, e a reprodução mostra em que direção.

Tabela 9: Coeficientes do ajuste a $\log_2 x + b$ da potência PKG ao número médio de núcleos ativos, no artigo, nos dados restritos a C++ e Python e na reprodução, com a razão entre a potência a dois núcleos e a um núcleo, que é o ganho mínimo de vazão para que paralelizar compense em energia.

Origem	a (W)	b (W)	R^2	Potência(2)/Potência(1)
artigo	31,00	246,00	0,72	1,13
restrito	37,53	212,85	0,83	1,18
reprodução	13,43	30,43	0,85	1,44

A curva captura a tendência geral, e a Figura 3 mostra dois desvios na reprodução: spectral-norm em Python, com 4 núcleos, fica na potência dos programas com 8, cerca de

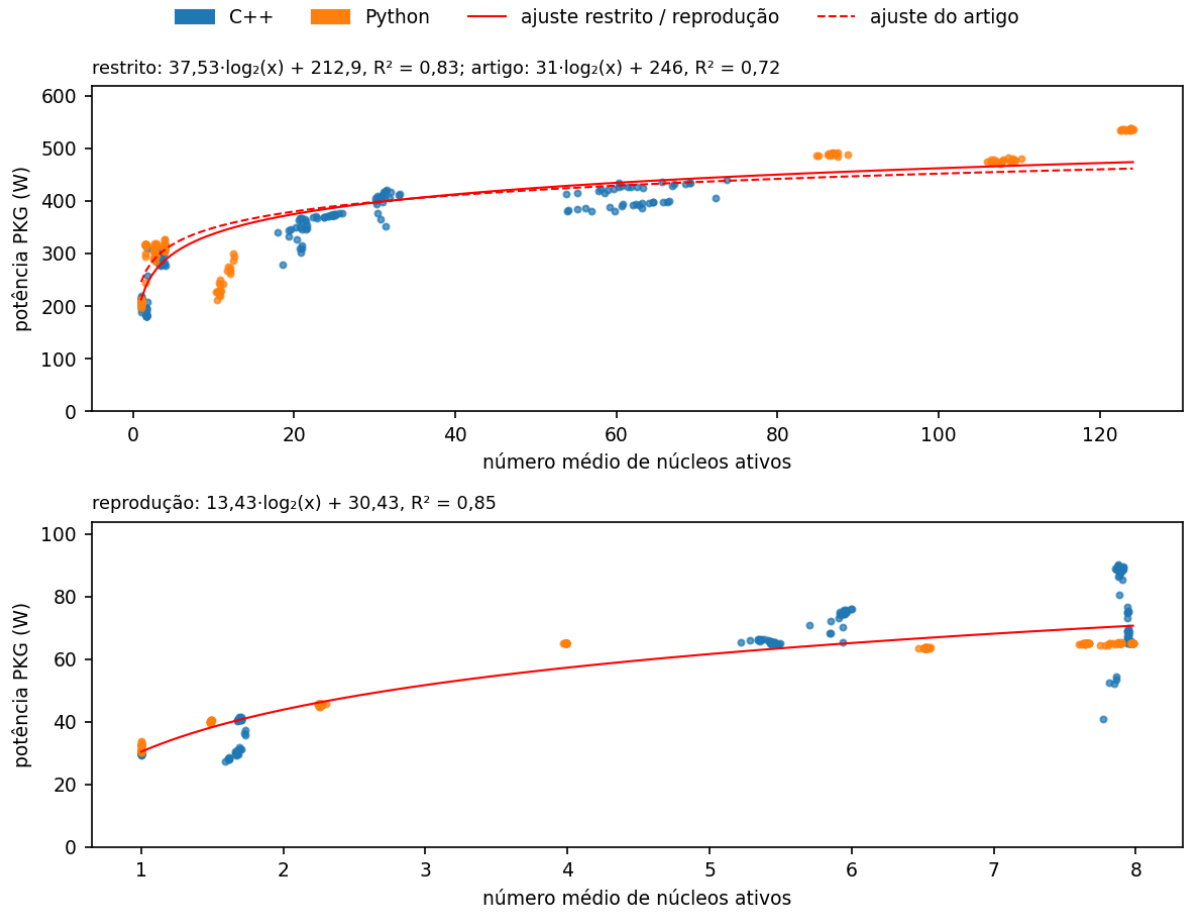


Figura 3: Potência PKG de cada execução na configuração padrão em função do número médio de núcleos ativos, no formato da Figura 8a do artigo: dados restritos a C++ e Python, com o ajuste a esses pontos e, tracejada, a curva publicada no artigo (acima), e reprodução com o ajuste próprio (abaixo).

7 W acima do ajuste; e, com 8 núcleos, os programas C++ se espalham verticalmente, porque a potência também depende do tipo de instrução executada. Com o mesmo número de núcleos, os pontos das duas linguagens se sobrepõem.

3.4 Energia de DRAM e atividade de memória

O segundo fator do modelo do artigo é a atividade de memória, medida pela taxa de faltas de LLC. A Figura 8b do artigo ajusta a potência de DRAM a essa taxa por uma reta, $1,68 \cdot 10^{-8} \cdot x + 12$ W, R^2 de 0,93, e observa que a DRAM fica entre 2% e 8% da energia de PKG. A Figura 4 e a Tabela 10 apresentam os resultados no mesmo formato, com as três linhas.

O ajuste da reprodução dá $0,80 \cdot 10^{-8} \cdot x + 0,57$ W, com R^2 de 0,996. O intercepto (0,57 W) é a potência de repouso da memória instalada; no servidor é 12 W. A inclinação (0,80) é o custo de cada acesso que sai da cache; no servidor é 1,68. Nos dados restritos o ajuste dá $2,44 \cdot 10^{-8} \cdot x + 11,75$ W, R^2 de 0,96: a inclinação é maior porque as duas linguagens cobrem uma faixa menor de faltas de LLC.

A fração DRAM/PKG na configuração padrão fica entre 0,6% e 4,5% na reprodução,

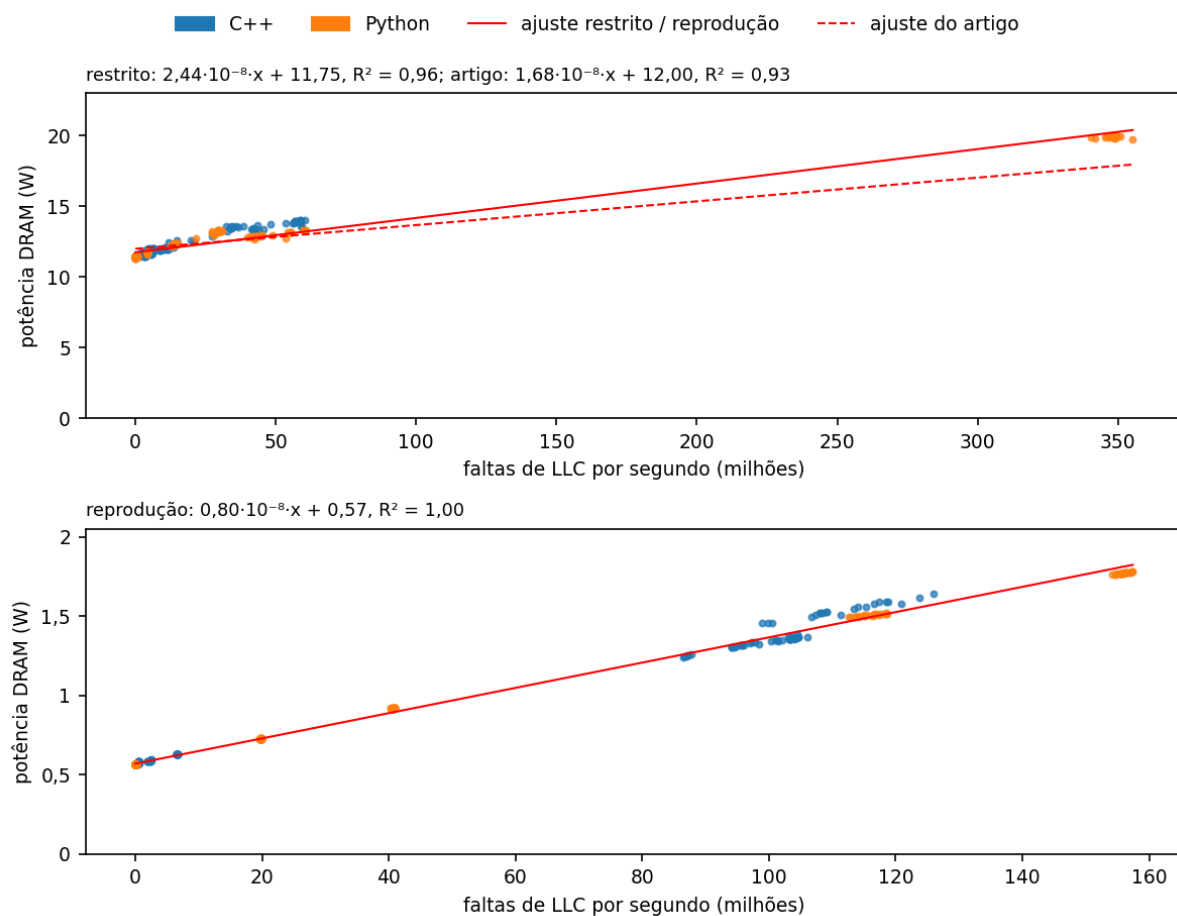


Figura 4: Potência de DRAM de cada execução na configuração padrão em função da taxa de faltas de LLC, no formato da Figura 8b do artigo: dados restritos a C++ e Python, com o ajuste a esses pontos e, tracejada, a reta publicada no artigo (acima), e reprodução com o ajuste próprio (abaixo).

Tabela 10: Coeficientes do ajuste a $x + b$ da potência de DRAM à taxa de faltas de LLC, com a em watts por 10^8 faltas por segundo, e faixa da razão DRAM/PKG entre execuções: artigo, dados restritos a C++ e Python e reprodução na configuração padrão; dados restritos e reprodução na frequência mínima.

Configuração	Origem	a	b (W)	R^2	DRAM/PKG mín. (%)	DRAM/PKG máx. (%)
padrão	artigo	1,68	12,00	0,93	2,00	8,00
	restrito	2,44	11,75	0,96	2,16	7,48
	reprodução	0,80	0,57	1,00	0,64	4,53
mín. freq.	restrito	—	—	—	6,34	7,05
	reprodução	—	—	—	30,83	44,90

contra 2% e 8% no artigo: a memória é secundária nos dois conjuntos. Na frequência mínima a fração sobe para 31% a 45% na reprodução, porque a potência do processador cai muito mais que a da memória; nos dados restritos ela fica entre 6,3% e 7,1%, porque o pacote do servidor, com 128 núcleos, continua consumindo muito mais que a memória. O peso da memória depende do quanto o processador consome, não do programa.

4 Comparação entre os artigos

As duas reproduções mediram as mesmas implementações, com o mesmo compilador e o mesmo interpretador, na mesma máquina, nos três estados do processador descritos na Seção 1.3. A comparação usa os oito benchmarks com as duas linguagens nos dois estudos, que excluem `fasta`, sem versão C++ no repositório de van Kempen et al. (2024), e `pidigits`, retirado nas duas reproduções. A grandeza comparada é a razão Python/C++ por benchmark. Como as duas linguagens são medidas na mesma máquina e no mesmo estado, a razão não depende do hardware, do domínio RAPL nem do método de agregação de cada seção.

As Tabelas 11 e 12 apresentam a razão de tempo e a de energia por benchmark em seis conjuntos: os dados restritos e a reprodução de Pereira et al. (2021), e os dados restritos e a reprodução de van Kempen et al. (2024) nas configurações padrão e de frequência mínima. A Figura 5 mostra quatro desses conjuntos.

Tabela 11: Razão Python/C++ de tempo por benchmark, nos oito benchmarks comuns: dados restritos e reprodução de Pereira et al. (2021), e dados restritos e reprodução de van Kempen et al. (2024) nas configurações padrão e de frequência mínima.

Benchmark	Pereira		van Kempen			
	restrito	reprodução	restrito padrão	restrito mín. freq.	reprodução padrão	reprodução mín. freq.
binary-trees	40,48	16,04	22,54	22,41	20,10	25,38
fannkuch-redux	45,70	24,87	7,12	17,15	27,92	27,54
k-nucleotide	10,21	11,07	2,99	16,11	12,79	14,64
mandelbrot	189,90	150,87	66,14	131,89	155,32	148,65
n-body	148,23	102,72	79,27	79,31	107,91	109,23
regex-redux	0,67	0,68	0,61	1,14	0,73	1,10
reverse-complement	6,71	5,63	2,35	5,35	5,14	7,03
spectral-norm	206,95	367,54	270,54	232,69	371,78	188,08

Tabela 12: Razão Python/C++ de energia por benchmark, nos mesmos conjuntos da Tabela 11.

Benchmark	Pereira		van Kempen			
	restrito	reprodução	restrito padrão	restrito mín. freq.	reprodução padrão	reprodução mín. freq.
binary-trees	43,50	20,79	15,26	22,43	17,21	24,75
fannkuch-redux	58,15	28,00	8,45	17,15	27,66	29,21
k-nucleotide	14,41	16,57	3,94	16,10	12,73	15,12
mandelbrot	240,01	180,11	83,20	132,44	147,25	160,37
n-body	137,45	113,63	77,99	79,43	110,99	114,67
regex-redux	0,92	0,85	0,64	1,14	0,81	1,14
reverse-complement	5,23	5,08	3,96	5,31	6,62	6,91
spectral-norm	372,81	322,63	219,77	232,83	273,35	217,81

A ordem dos benchmarks praticamente não muda entre os conjuntos. Em todos eles, `spectral-norm`, `mandelbrot` e `n-body` têm as maiores razões, e `regex-redux` e

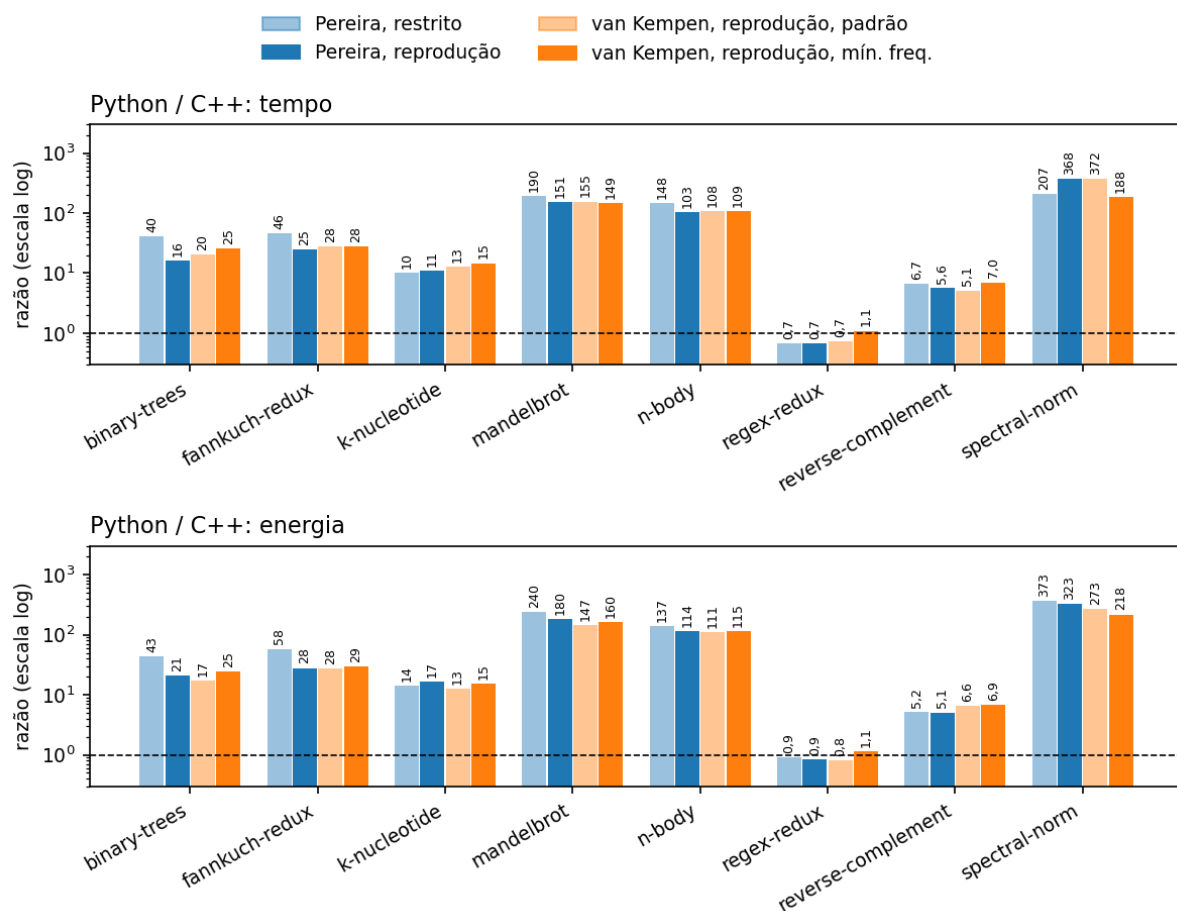


Figura 5: Razões Python/C++ de tempo e de energia por benchmark, em escala logarítmica: dados restritos e reprodução de Pereira et al. (2021), e reprodução de van Kempen et al. (2024) nas configurações padrão e de frequência mínima. Os três últimos conjuntos foram medidos na mesma máquina.

reverse-complement, as menores. regex-redux é o único benchmark com razão abaixo de 1 em algum conjunto, isto é, o único em que Python chega a ser mais rápido e mais econômico que C++.

Nos três conjuntos medidos na máquina de reprodução, a razão de cada benchmark fica próxima nos três estados do processador (Figura 5). Com o mesmo compilador e o mesmo interpretador, mudar a frequência e o número de núcleos disponíveis não altera a distância entre as linguagens. A razão muda entre os estados apenas nos benchmarks cujas versões ocupam números diferentes de núcleos (Tabela 1). Em spectral-norm, em que a versão Python ocupa metade dos núcleos, a razão de tempo é de 188 na frequência mínima e de 368 e 372 com todos os núcleos disponíveis. Em regex-redux, Python é mais rápido com todos os núcleos (razões de 0,68 e 0,73) e mais lento em um núcleo (1,10).

Nos dados restritos de Pereira et al. (2021), a razão é maior que na reprodução do mesmo artigo em cinco dos oito benchmarks, como binary-trees (40,5 contra 16,0) e fannkuch-redux (45,7 contra 24,9). A diferença acompanha o que a Seção 2.2 mostrou: o interpretador passou de CPython 3.6.1 para 3.12.6, e o tempo de Python caiu mais que o de

C++.

Nos dados restritos de van Kempen et al. (2024) na configuração padrão, a razão cai em três benchmarks, `fannkuch-redux`, `k-nucleotide` e `mandelbrot`, cujas razões de tempo são de 7,1, 3,0 e 66 na configuração padrão e sobem para 17, 16 e 132 quando o mesmo servidor executa em um único núcleo (Tabela 11). Nos três benchmarks as duas versões são paralelas (Tabela 1). Com 128 núcleos disponíveis, a versão Python ganha mais com o paralelismo que a versão C++, e a razão cai.

Duas consequências decorrem desses resultados. A primeira é que a razão entre Python e C++ na configuração padrão depende de quantos núcleos a máquina oferece, porque as implementações escalam de forma diferente. No desktop de oito núcleos as duas versões ficam próximas da saturação (Tabela 1), e no servidor de 128 núcleos a razão cai. É a anomalia de concorrência que van Kempen et al. (2024) descrevem para JavaScript e TypeScript, e ela aparece nos dados do próprio artigo. A segunda é que o fator publicado por Pereira et al. (2021), de 56,6 em energia e 46,1 em tempo em relação a C++ (Tabela 6), não é uma propriedade das linguagens. Com as mesmas implementações, a razão de um mesmo benchmark varia conforme o interpretador, o número de núcleos disponíveis e o estado do processador, como `fannkuch-redux`, de 7,1 a 45,7 no tempo (Tabela 11), fatores que o protocolo do artigo não controla.

5 Conclusão

As duas reproduções confirmam os problemas que van Kempen et al. (2024) apontam no trabalho de Pereira et al. (2021) e mostram que o modelo proposto pelo segundo artigo se sustenta, com coeficientes que dependem da máquina.

Do trabalho de Pereira et al. (2021), a reprodução mantém a ordem e não a magnitude. C++ é mais rápido e mais econômico que Python em oito dos nove benchmarks, e a ordem dos benchmarks pela razão Python/C++ é a mesma nos dados restritos e na reprodução. O fator entre as linguagens, porém, cai de 56,6 para 32,8 em energia com as mesmas implementações (Tabela 6), porque depende da versão do interpretador, do número de núcleos ocupados e do estado do processador, que o protocolo do artigo não controla. Os pontos em que a análise do artigo falha são os que van Kempen et al. (2024) indicam: o medidor produz leituras negativas por estouro do contador, as implementações ocupam números diferentes de núcleos, e a variação da razão energia/tempo que o artigo atribui à linguagem acompanha esse paralelismo. No par C++ e Python, a linguagem mais rápida é sempre a mais econômica.

Do trabalho de van Kempen et al. (2024), a reprodução mantém o modelo e não os coeficientes. Com frequência e número de núcleos fixos, a potência é a mesma para todos os pares a menos de 5%, e a energia acompanha o tempo, como no artigo. O número de núcleos ativos e a taxa de faltas de LLC governam a potência com as mesmas formas do artigo, logarítmica e linear, mas os coeficientes são os da máquina: no desktop, paralelizar compensa

em energia a partir de 44% de ganho de vazão, e não de 13%, e o peso da DRAM, secundário na configuração padrão, sobe para um terço da energia na frequência mínima. A reprodução encontra ainda, nos dados publicados pelos autores, o defeito de *pidigits*, que não altera seus resultados de potência.

Tomadas em conjunto, as reproduções mostram que a razão entre as linguagens é estável quando o interpretador e o paralelismo estão fixos. Na mesma máquina, nos três estados do processador, a razão de cada benchmark praticamente não muda, exceto onde as versões das duas linguagens ocupam números diferentes de núcleos. A razão sobe com o interpretador mais antigo, nos dados de Pereira et al. (2021), e cai no servidor de 128 núcleos, onde as versões Python escalam mais que as C++. Esse resultado sustenta a conclusão de van Kempen et al. (2024), de que a linguagem afeta a energia por meio do tempo de execução, e não sustenta o uso do fator publicado por Pereira et al. (2021) como medida da diferença entre linguagens.

O trabalho tem limitações que delimitam o que esses resultados sustentam. A restrição a C++ e Python deixa fora as anomalias entre linguagens próximas que motivam van Kempen et al. (2024), e a ausência de C impede verificar a diferença entre C e C++ reportada por Pereira et al. (2021). As medições vêm de uma única máquina, e os coeficientes obtidos, como o limiar de 44% para paralelizar, valem para ela. Com oito ou nove benchmarks comuns, as comparações se apoiam em poucos pares. Nenhuma das campanhas verificou a saída dos programas, o que custou *pidigits* nas duas reproduções, e *fasta* ficou fora da comparação por não ter versão C++ no repositório de van Kempen et al. (2024). Por fim, as reproduções seguem os artefatos publicados com o mínimo de alteração; um protocolo próprio, com verificação de saída, controle explícito de frequência e mais repetições, é o passo seguinte.

Referências

- Isaac Gouy. The computer language benchmarks game. <https://benchmarksgame-team.pages.debian.net/benchmarksgame/>, 2026. Acesso em setembro de 2026.
- Rui Pereira, Marco Couto, Francisco Ribeiro, Rui Rua, Jácome Cunha, João Paulo Fernandes e João Saraiva. Ranking programming languages by energy efficiency. *Science of Computer Programming*, 205:102609, 2021. doi: 10.1016/j.scico.2021.102609.
- Nicolas van Kempen, Hyuk-Jin Kwon, Dung Tuan Nguyen e Emery D. Berger. It's not easy being green: On the energy efficiency of programming languages. arXiv:2410.05460, 2024.